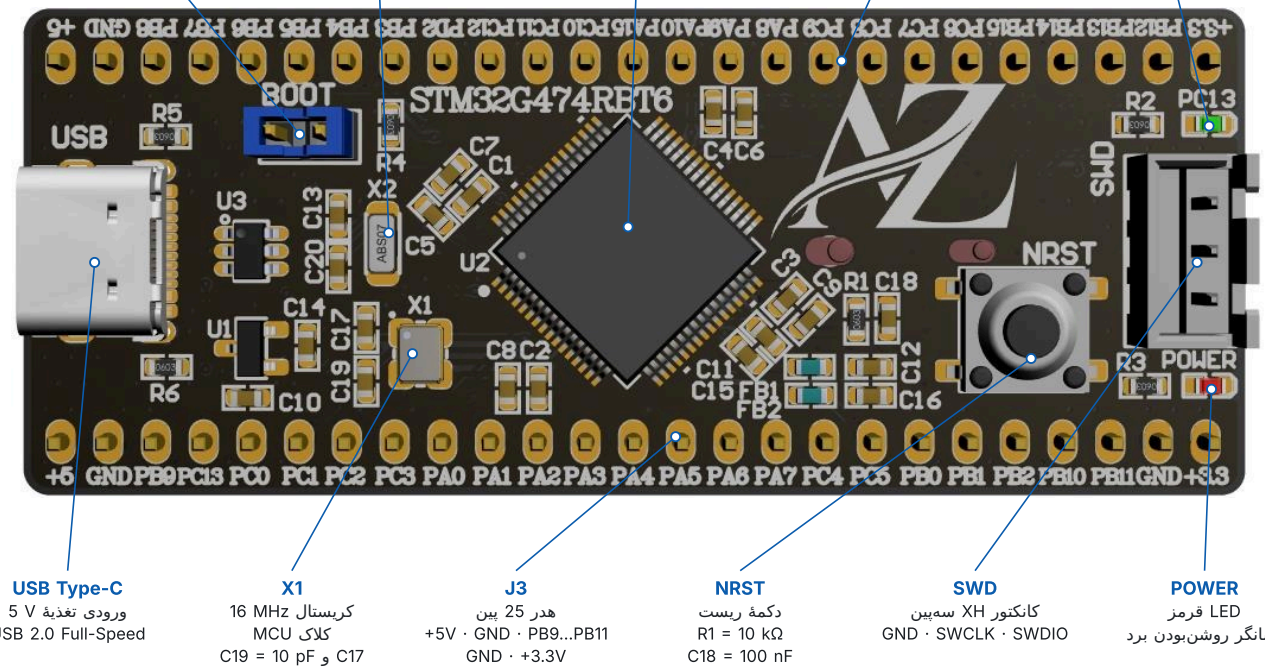
راهنمای کاربر AH_G474

شروع به کار، راهنمای سخت‌افزار و نکات طلایی برای برد هدر STM32G474RBT6: تغذیه، کلاک‌ها، بوت، برنامه‌ریزی، USB، پایه‌های I/O و همهٔ پریفرال‌های مهم، متناسب با سیم‌کشی همین برد

5 گام	170 MHz	50 mA	SWD + USB
از کابل USB تا چشمک‌زدن LED	دستور ساخت PLL برای کریستال 16 MHz	حد جریان رگولاتور 3.3 V	دو راه برای برنامه‌ریزی و دیباگ



شکل 1. نمای بالای برد AH_G474 همراه با کنترل‌ها و کانکتورهایی که در این راهنما به کار می‌روند. نام قطعات مطابق سیلک‌اسکرین و شماتیک است.

فهرست مطالب

1. مقدمه 2
2. ایمنی و محدودیت‌ها 2
3. شروع سریع 3
4. معرفی برد 3
5. تغذیه 4
6. کلاک‌ها 4
7. ریست، بوت و برنامه‌ریزی 5
9. هدرهای I/O 7
10. راهنمای پریفرال‌ها 9
10.6 تایمرها و PWM 12
10.10 کامپاراتورها، آپ‌امپ‌ها و شتاب‌دهنده‌های ریاضی 15
10.11 فریمور دستگاه USB 15
10.12 واچ‌داگ‌ها و مدیریت خطا 15
10.13 کم‌مصرفی و بیدارکردن 16
12. چک‌لیست نکات طلایی 17
13. عیب‌یابی 17
14. مراجع و تاریخچهٔ نسخه‌ها 18

1. مقدمه

این راهنما نشان می‌دهد چگونه از برد هدر AH_G474 استفاده کنید و بیشترین بهره را از STM32G474RBT6 روی آن ببرید. اطلاعات برد از دیتاشیت AH_G474 (نسخهٔ V0.1) گرفته شده و نکات میکروکنترلر مطابق سیم‌کشی همین برد تنظیم شده‌اند.

نحوهٔ مطالعهٔ راهنما

- **اطلاعات برد** (قطعات، مقادیر، پایه‌ها) از دیتاشیت AH_G474 و بخش‌های شماتیک آن آمده است.
- **اطلاعات میکروکنترلر** از دیتاشیت STM32G474، RM0440 و AN2606 شرکت ST گرفته شده؛ آخرین نسخهٔ آن‌ها را بررسی کنید.
- کادرهای ★ **نکتهٔ طلایی** توصیه‌های تخصصی مرتبط با سیم‌کشی همین برد هستند؛ کادرهای **هشدار** مواردی را نشان می‌دهند که می‌توانند به برد آسیب بزنند.

به چه چیزهایی نیاز دارید

- کابل USB Type-C داده‌دار (کابل‌های فقط‌شارژ شناسایی نمی‌شوند).
- یک پروب SWD (مثلاً ST-LINK) همراه با کابل XH سه‌پین، یا فقط کابل USB و جامپر BOOT (بخش 7.4)؛ به‌علاوهٔ STM32CubeIDE و STM32CubeProgrammer از ST.

2. ایمنی و محدودیت‌ها

محدودیت	مقدار	چرا مهم است
جریان کشیده‌شده از برد	**≤ 50 mA**	جریان بیشتر به رگولاتور 3.3 V (AP2120N-3.3) آسیب می‌زند؛ میکروکنترلر و LEDها هم از همین رگولاتور تغذیه می‌شوند.
سطح منطقی، تغذیهٔ MCU	**3.3 V**	همهٔ I/O هدرها 3.3 V است؛ پیش از اتصال سیگنال 5 V بخش 9.3 را بخوانید.
تغذیهٔ برد	**5 V از USB Type-C**	ریل 5 V روی پایه‌های +5V هم در دسترس است.
PB8 و جامپر BOOT	**متصل به +3.3V**	تا وقتی جامپر نصب است، هرگز PB8 را به سطح پایین نکشید (بخش 7.2).
باتری پشتیبان CR1220	**سلول یک‌بارمصرف**	شارژپذیر نیست: هرگز شارژ VBAT را در فریمور فعال نکنید (بخش 11).

بیش از 50 mA از برد نکشید. این کار به رگولاتور آسیب می‌زند. هرگز منبع تغذیه‌ای به پایه‌های +3.3V وصل نکنید و تا وقتی USB وصل است پایه‌های +5V را تغذیه نکنید: مسیر تغذیه یا حفاظت در برابر برگشت جریان در مستندات برد توضیح داده نشده است.

3. شروع سریع

1. **جامپر BOOT را بررسی کنید.** برای راه‌اندازی عادی از Flash آن را باز بگذارید (PB8 در این حالت با R4 پایین کشیده می‌شود).
2. **برد را روشن کنید.** کابل USB Type-C داده‌دار را به کانکتور لبهٔ چپ وصل کنید. LED قرمز POWER روشن می‌شود.
3. **پروب SWD را وصل کنید.** پروب را به کانکتور XH سه‌پین وصل کنید: پین 1 GND، پین 2 SWCLK، پین 3 SWDIO (بخش 7.3). پروب ندارید؟ به‌جای آن از USB DFU استفاده کنید (بخش 7.4).
4. **پروژه را بسازید.** در STM32CubeIDE قطعهٔ `STM32G474RBTx` را انتخاب کنید. گزینهٔ *Debug* → *SYS* را روی *Serial Wire* بگذارید، PC13 را به‌صورت *GPIO_Output* تنظیم و کد را تولید کنید. فعلاً کلاک را در حالت پیش‌فرض بگذارید.
5. **LED را چشمک بزنید.** خطوط زیر را در حلقهٔ اصلی بنویسید، بیلد و اجرا کنید. LED سبز روی PC13 چشمک می‌زند (PC13 بالا = LED روشن).

```
while (1)
{
  HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);   /* PC13: green user LED, high = on */
  HAL_Delay(500);
}
```

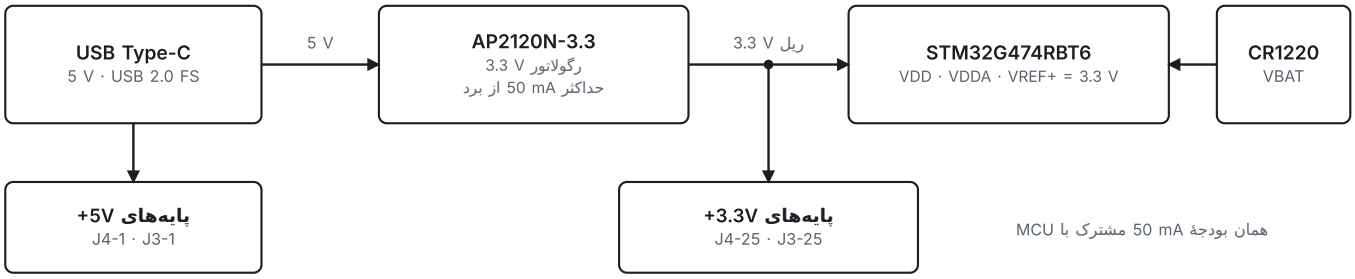
> ★ **نکتهٔ طلایی.** اولین چشمک را با کلاک پیش‌فرض بزنید. پس از ریست، میکروکنترلر با نوسان‌ساز داخلی 16 MHz (HSI16) کار می‌کند. پیش از تنظیم PLL ابتدا LED را به چشمک بیندازید: این کار سالم‌بودن تغذیه، SWD و ابزارهای شما را ثابت می‌کند. بعد به دستور 170 MHz در بخش 6.2 بروید.

4. معرفی برد

قطعه	محل	چیست و چگونه استفاده می‌شود
USB Type-C	لبهٔ چپ، سمت بالا	ورودی تغذیهٔ 5 V و درگاه دستگاه USB 2.0 Full-Speed (بخش 8).
POWER LED	قرمز، لبهٔ پایین‌راست	تا وقتی برد روشن است، روشن می‌ماند.
PC13 LED	سبز، لبهٔ بالاراست	LED کاربر: PC13 → R2 (1 kΩ) → LED → GND. PC13 بالا = روشن.
NRST	دکمه، سمت راست MCU	میکروکنترلر را ریست می‌کند (R1 بالاکش 10 kΩ، C18 = 100 nF به GND).
BOOT	جامپر، بالاچپ کنار USB	نصب‌شده: PB8 = 3.3 V (انتخاب بوت، 7.2). باز: PB8 با R4 (10 kΩ) پایین کشیده می‌شود.
SWD	کانکتور XH سه‌پین، لبهٔ راست	پین 1 GND، پین 2 SWCLK، پین 3 SWDIO. همین سیگنال‌ها در سمت پایین با برچسب DIO / CLK / GND مشخص شده‌اند.
J4 · J3	لبه‌های بالا و پایین	دو هدر 25 پینی با فاصلهٔ 2.54 mm شامل 43 GPIO، +5V، +3.3V و GND (پین‌اوت در 9.1).
X1 · X2	کنار MCU	کریستال 16 MHz (کلاک MCU) و کریستال 32.768 kHz (کلاک RTC).
CR1220	جایگاه باتری، سمت پایین	باتری پشتیبان RTC روی VBAT (بخش 11).

5. تغذیه

5.1 مسیر تغذیه



شکل 2. مسیر تغذیه. ولتاژ 5 V از USB Type-C وارد می‌شود؛ AP2120N-3.3 ریل 3.3 V را می‌سازد؛ باتری CR1220 فقط VBAT را تغذیه می‌کند.

ولتاژ 5 V از کانکتور USB Type-C وارد می‌شود و به پایه‌های 5V+ هم می‌رسد. رگولاتور AP2120N-3.3 ریل 3.3 V را برای میکروکنترلر، مرجع ADC و پایه‌های 3.3V+ می‌سازد. باتری CR1220 فقط پایهٔ VBAT میکروکنترلر را تغذیه می‌کند.

5.2 بودجهٔ توان

حد اعلام‌شده در دیتاشیت، 50 mA جریان کشیده‌شده از برد است. میکروکنترلر، LEDها و هر چیزی که به پایه‌های 3.3V+ وصل کنید از یک رگولاتور تغذیه می‌شوند؛ بنابراین جریان باقی‌مانده برای مدارهای شما برابر است با 50 mA منهای جریانی که خود برد مصرف می‌کند.

مصرف‌کننده	مقدار معمول	توضیح
STM32G474RBT6	وابسته	به سرعت کلاک و پریفرال‌های فعال بستگی دارد؛ در 170 MHz می‌تواند به ده‌ها میلی‌آمپر برسد (جدول‌های IDD در دیتاشیت ST را ببینید).
PC13 LED	≈ 1 mA	فقط وقتی روشن است: حدود 1 kΩ / (3.3 V − 2 V) برای LED سبز.
POWER LED	کم	همیشه روشن است؛ مقدار مقاومت آن در بخش‌های دیتاشیت داده نشده است.
مدارهای شما	باقی‌مانده	50 mA منهای ردیف‌های بالا.

گرما: رگولاتور LDO اختلاف ولتاژ را به گرما تبدیل می‌کند: P = (5 V − 3.3 V) × I = 1.7 V × I. در 50 mA این مقدار حدود 85 mW است که همگی در بستهٔ کوچک رگولاتور تلف می‌شود.

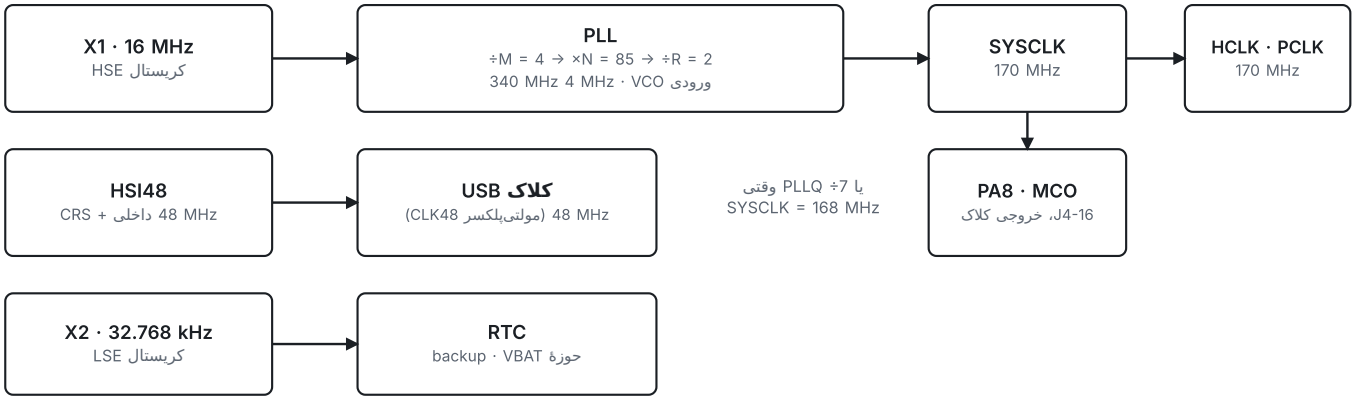
★ **نکتهٔ طلایی.** به‌جای حدس‌زدن، اندازه بگیرید. یک توان‌سنج USB-C بین رایانه و برد بگذارید و جریان را با فریمور واقعی خودتان بخوانید. اگر از همین حالا نزدیک 40 mA است، بار اضافه نکنید؛ برای آزادکردن بودجه، SYSCLK را کم کنید یا در حلقه‌های بیکار از حالت Sleep استفاده کنید.

★ **نکتهٔ طلایی.** موتور، رله، نوار LED و نمایشگرهای دارای نور پس‌زمینه تغذیهٔ جداگانه می‌خواهند. زمین‌ها را به هم وصل کنید (هر پایهٔ GND روی J4 یا J3) و هرگز آن‌ها را از پایه‌های 3.3V+ تغذیه نکنید.

6. کلاک‌ها

6.1 منابع کلاک

روی برد یک کریستال 16 MHz به نام X1 (HSE)، خازن‌های بار C17 و C19 برابر 10 pF) برای MCU و یک کریستال 32.768 kHz به نام X2 (LSE، خازن‌های C13 و C20 برابر 22 pF) برای RTC وجود دارد. HSI16 (کلاک ریست)، HSI48 (همراه با CRS می‌تواند کلاک USB را بدهد) و LSI (واچ‌داگ مستقل) درون میکروکنترلر هستند.



شکل 3. درخت کلاک برای 170 MHz با کریستال 16 MHz. USB از HSI48 قفل‌شده با CRS یا از PLLQ در 168 MHz کار می‌کند.

6.2 دستور STM32CubeMX برای 170 MHz

تنظیم	مقدار
RCC → HSE · LSE	هر دو Crystal/Ceramic Resonator (LSE فقط اگر از RTC استفاده می‌کنید)
PLL · M · N · R منبع	HSE · ÷4 · ×85 · ÷2 → SYSCLK = 170 MHz
AHB · APB1 · APB2 تقسیم‌کننده‌های	÷1 · ÷1 · ÷1
Flash مقیاس ولتاژ · تأخیر	Range 1 boost mode · 8 wait state (CubeMX هر دو را در 170 MHz تنظیم می‌کند)
USB کلاک (CLK48)	HSI48، با CRS فعال و SYNC source = USB
Flash	کش دستورالعمل، کش داده و prefetch فعال
SYS → Debug	Serial Wire (هرگز No Debug نه)

★ **نکتهٔ طلایی.** USB به کلاک 48 MHz با دقت حدود ±0.25 % نیاز دارد. در 170 MHz، PLL نمی‌تواند آن را از کریستال 16 MHz بسازد (340 MHz ÷ 7 = 48.57 MHz، یعنی 1.2 % خطا). از HSI48 قفل‌شده با CRS روی شروع فریم USB استفاده کنید، یا SYSCLK را روی 168 MHz بگذارید (M = 4, N = 84, R = 2) و PLLQ = ÷7 را بگیرید تا دقیقاً 48 MHz شود.

★ **نکتهٔ طلایی.** بار کریستال. C17 و C19 با احتساب ظرفیت پارازیتی برای X1 حدود 8 تا 10 pF بار می‌سازند؛ C13 و C20 برای X2 حدود 14 تا 16 pF. اگر کریستال را عوض می‌کنید، با ظرفیت بار مشخص‌شدهٔ آن تطبیق دهید؛ اگر LSE دیر استارت می‌شود، سطح درایو آن (LSEDRV) را بالا ببرید.

★ **نکتهٔ طلایی.** کلاک‌ها را بدون اسیلوسکوپ بررسی کنید: SYSCLK یا HSE را به خروجی MCO روی PA8 (پین J4 16) بدهید و اندازه بگیرید. مطمئن شوید HSE_VALUE در تنظیمات HAL برابر 16000000 است، وگرنه baud rateها و زمان‌بندی‌ها غلط می‌شوند.

★ **نکتهٔ طلایی.** راه‌اندازی 170 MHz را دستی ننویسید: از کد HAL که CubeMX می‌سازد استفاده کنید (مقیاس ولتاژ، wait stateها و تقسیم‌کنندهٔ میانی AHB از یک ترتیب ثابت پیروی می‌کنند). برای محصول، CSS (Clock Security System) را روی HSE فعال کنید: اگر کریستال از کار بیفتد، MCU به HSI16 برمی‌گردد و NMI ایجاد می‌کند.

7. ریست، بوت و برنامه‌ریزی

7.1 ریست

دکمهٔ NRST خط ریست را به GND می‌کشد؛ R1 (10 kΩ) آن را بالا نگه می‌دارد و C18 (100 nF) آن را فیلتر می‌کند. کانکتور SWD پین NRST ندارد، پس پروب نمی‌تواند برد را با سیم ریست کند (بخش 7.3 را ببینید).

7.2 انتخاب بوت (جامپر BOOT، پایهٔ PB8)

جامپر BOOT	PB8	منبع بوت با option byteهای پیش‌فرض
باز	پایین (R4)	Flash اصلی: برنامهٔ شما اجرا می‌شود.
نصب‌شده	بالا (3.3 V)	حافظهٔ سیستم: بوت‌لودر داخلی اجرا می‌شود (USB DFU، USART).

PB8 همان پایهٔ BOOT0 میکروکنترلر STM32G474RBT6 است. سطح بوت هنگام ریست خوانده می‌شود؛ جدول دقیق بوت به option byteها (nSWBOOT0، nBOOT0، nBOOT1) بستگی دارد و در RM0440 تعریف شده است.

تا وقتی جامپر BOOT نصب است، PB8 را به سطح پایین نکشید. جامپر، PB8 را مستقیماً به +3.3V وصل می‌کند. همچنین option byteهای nSWBOOT0 و nBOOT0 را روی مقدار پیش‌فرض بگذارید: تغییر آن‌ها می‌تواند جامپر را بی‌اثر کند.

★ **نکتهٔ طلایی.** PB8 یک پایهٔ I/O هم هست (I²C1_SCL، FDCAN1_RX، TIM16_CH1، TIM4_CH3)، اما روی این برد همیشه مقاومت پایین‌کش 10 kΩ (R4) را دارد و با جامپر نصب‌شده، مستقیماً 3.3 V است. از PB8 برای چیزی که باید هنگام ریست high-impedance باشد استفاده نکنید. به‌جای آن برای I²C1 از PB6 / PB7 و برای FDCAN از FDCAN2 (PB6 / PB5) استفاده کنید.

7.3 اتصال SWD



شکل 4. سیم‌کشی SWD. کانکتور GND، SWCLK و SWDIO را دارد؛ برای SWO یک سیم اضافه از PB3 لازم است.

پین	سیگنال	برچسب پد در سمت پایین	پین پروب
1	GND	GND	GND
2	SWCLK	CLK	SWCLK / TCK
3	SWDIO	DIO	SWDIO / TMS

★ **نکتهٔ طلایی.** printf با SWO. کانکتور پین SWO ندارد. برای ردیابی printf / ITM، پایهٔ PB3 (پین J4 8) را به پین SWO پروب وصل کنید و SWO را در پیکربندی دیباگ فعال کنید.

★ **نکتهٔ طلایی.** اتصال تحت ریست را دستی انجام دهید. چون سیم NRST نیست، دکمهٔ NRST را نگه دارید، در STM32CubeProgrammer روی Connect بزنید و بعد رها کنید. با سیم‌های بلند از کلاک SWD پایین (1 تا 4 MHz) استفاده کنید و سیم GND را کنار SWCLK بکشید. در حالت‌های Stop یا Standby ممکن است دیباگر قطع شود، مگر اینکه DBG_STANDBY / DBG_STOP در DBGMCU_CR تنظیم شده باشند.

7.4 USB DFU (بدون نیاز به پروب)

1. **جامپر BOOT را نصب کنید.**
2. **USB را وصل کنید** (یا اگر وصل است NRST را بزنید). میکروکنترلر بوت‌لودر داخلی را اجرا می‌کند.
3. **STM32CubeProgrammer را باز کنید،** USB را انتخاب و Connect را بزنید.

4. **برنامه‌ریزی کنید:** فایل .hex، .bin یا .elf را بنویسید (Flash از 0x08000000 شروع می‌شود) و برنامه را اجرا کنید.
5. **جامپر را بردارید** و برای اجرا از Flash، NRST را بزنید.

بوت‌لودر ROM را از طریق USART1 هم می‌توان فعال کرد (PA9 / PA10، پین‌های 14 / 15 J4)؛ برای رابط‌های STM32G47x به AN2606 نگاه کنید.

7.5 بازیابی و حفاظت

- **از SWD بیرون ماندید؟** فریموری که پایه‌های دیباگ را جابه‌جا کند یا بیش از حد عمیق بخوابد می‌تواند پروب را قفل کند. جامپر BOOT را نصب کنید، از راه USB وصل شوید (7.4) و پاک‌سازی کامل تراشه (full chip erase) انجام دهید.
- **حفاظت خواندن (RDP):** Level 1 دسترسی دیباگ را می‌بندد و برگشت از آن Flash را پاک می‌کند؛ Level 2 دائمی است. هرگز Level 2 را روی برد توسعه تنظیم نکنید.

8. USB Type-C

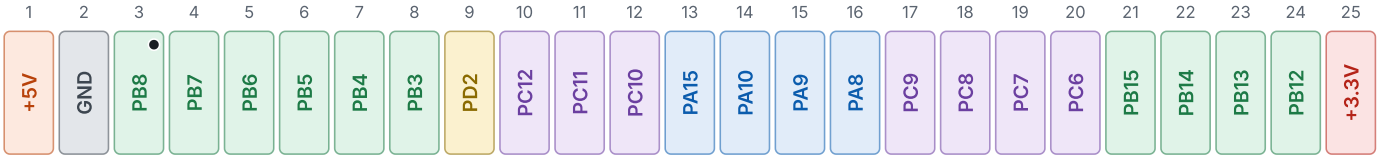
- این کانکتور تغذیهٔ 5 V و یک درگاه **دستگاه USB 2.0 Full-Speed** فراهم می‌کند. پایه‌های داده USB به میکروکنترلر وصل‌اند و روی هدرها نیستند.
- از کابل داده‌دار استفاده کنید. اگر کابل USB-C به USB-C از شارژر برق نمی‌دهد، کابل USB-A به USB-C یا درگاه رایانه را امتحان کنید.
- در CubeMX گزینهٔ *Device → USB (FS)* و یک کلاس مثل CDC (پورت COM مجازی) را فعال کنید. کلاک USB را مطابق 6.2 تنظیم کنید. نکات فریمور در 10.11 آمده است.

> ★ **نکتهٔ طلایی.** بدون بیرون‌کشیدن کابل، دوباره شناسایی کنید. پریفرال USB در STM32G4 یک پول‌آپ داخلی +D دارد که با نرم‌افزار کنترل می‌شود. در فریمور HAL_PCD_DevDisconnect() را صدا بزنید، 100 تا 200 ms صبر کنید و بعد HAL_PCD_DevConnect(): رایانه جدا و وصل‌شدن را می‌بیند و پس از هر دانلود لازم نیست به کابل دست بزنید.

9. هدرهای I/O

9.1 پین‌اوت

J4 · لبهٔ بالا · 25 پین







J3 · لبهٔ پایین · 25 پین

+5V · +3.3V · GND · پورت A · پورت B · پورت C · پورت D · PB8 و PC13 مشترک با قطعهٔ روی برد:

شکل 5. پین‌اوت هدرها از نمای بالای برد؛ پین 1 در سمت چپ (سمت USB Type-C). نقطه دو پایه‌ای را نشان می‌دهد که با قطعهٔ روی برد مشترک‌اند.

9.2 پایه‌های ویژه

پایه	هدر	نقش ویژه	نکته
PB8	J4-3	BOOT0، جامپر، R4	بخش 7.2 را ببینید. با جامپر نصب‌شده مستقیماً 3.3 V است.
PC13	J3-4	LED کاربر، 1 kΩ = R2	هنگام روشن‌بودن حدود 1 mA. PC13 در حوزهٔ backup قرار دارد: آن را خروجی کند و کم‌جریان بدانید (یادداشت‌های پایه در دیتاشیت ST را ببینید).
PA15 · PB4	J4-13 · 7	JTDI · NJTRST	هنگام ریست عملکرد جایگزین دیباگ دارند (پول‌آپ داخلی). وقتی فریمور آن‌ها را تنظیم کند، مثلاً با Debug = Serial Wire، به GPIO معمولی تبدیل می‌شوند.
PB3	J4-8	JTDO / SWO	برای ردیابی SWO (7.3) یا به‌عنوان GPIO استفاده کنید.
PA8	J4-16	MCO	خروجی کلاک برای اندازه‌گیری (6.2).
PA4 · PA5	J3-13 · 14	خروجی‌های DAC1	خروجی‌های آنالوگ (10.8 را ببینید).

9.3 نکات الکتریکی

- **منطق 3.3 V.** برد سطح‌ساز اضافه نکرده است. بسیاری از پایه‌های STM32G4 (نه همه) در حالت ورودی دیجیتال تحمل 5 V دارند، اما در حالت آنالوگ نه: نوع پایه (FT، FT_a، ...) را در دیتاشیت ST بررسی کنید یا از مبدل سطح ولتاژ یا مقاومت سری استفاده کنید.
- **پایه‌های استفاده‌نشده:** آن‌ها را در حالت آنالوگ بگذارید تا نشتی و نویز کم شود.
- **جریان خروجی:** دیتاشیت جریان هر پایه و کل بسته را محدود کرده است. برای موتور، رله و نوار LED از ترانزیستور یا IC درایور استفاده کنید.
- **خطوط هدر مستقیم به MCU می‌روند** (در بخش شماتیک مقاومت سری یا قطعهٔ ESD دیده نمی‌شود). کابل‌های بلند را با مقاومت سری و TVS یا کلمپ محافظت کنید.

> ★ **نکتهٔ طلایی.** پایه‌ها را پیش از سیم‌کشی برنامه‌ریزی کنید. بسیاری از عملکردها پایه مشترک دارند (PA9 هم USART1_TX، هم I²C2_SCL و هم TIM1_CH2 است). ابتدا در CubeMX پریفرال‌ها را انتخاب کنید، بگذارید تداخل‌ها را حل کند، بعد فهرست پایه‌ها را چاپ و از روی آن سیم‌کشی کنید. FDCAN2 (PB6 / PB5) و I²C1 (PB7 / PB6) را ترجیح دهید تا از پایهٔ BOOT یعنی PB8 دور بمانید.

9.4 راهنمای مسیردهی پریفرال‌ها

پریفرال	سیگنال‌ها ← پایه‌های هدر	توضیحات
USART1	TX PA9 (J4-15) · RX PA10 (J4-14)	جایگزین‌ها: PB7 / PB6، PC5 / PC4.
USART2	TX PA2 (J3-11) · RX PA3 (J3-12)	
USART3	TX PB10 (J3-22) · RX PB11 (J3-23)	جایگزین: PC11 / PC10.
UART4 · UART5	UART4 TX PC10 (J4-12) · RX PC11 (J4-11) UART5 TX PC12 (J4-10) · RX PD2 (J4-9)	UART4 با SPI3 و USART3 پایه مشترک دارد.
LPUART1	TX PC1 (J3-6) · RX PC0 (J3-5)	UART کم‌مصرف.
I²C1	SCL PB6 (J4-5) · SDA PB7 (J4-4)	PB8 / PB9 هم کار می‌کنند، اما PB8 پایهٔ BOOT است.
I²C2 · I²C3	I²C2 SCL PA9 (J4-15) · SDA PA8 (J4-16) I²C3 SCL PC8 (J4-18) · SDA PC9 (J4-17)	I²C پول‌آپ خارجی می‌خواهد (روی هدرها پول‌آپی ذکر نشده است).
SPI1	SCK PA5 (J3-14) · MISO PA6 (J3-15) · MOSI PA7 (J3-16) · NSS PA4 (J3-13)	
SPI2	SCK PB13 (J4-23) · MISO PB14 (J4-22) · MOSI PB15 (J4-21) · NSS PB12 (J4-24)	

پریفرال	سیگنال‌ها ← پایه‌های هدر	توضیحات
SPI3	SCK PC10 (J4-12) · MISO PC11 (J4-11) · MOSI PC12 (J4-10) · NSS PA15 (J4-13)	
FDCAN	FDCAN2 RX PB5 (J4-6) · TX PB6 (J4-5) (توصیه‌شده) FDCAN1 RX PB8 (J4-3) · TX PB9 (J3-3) FDCAN3 RX PB3 (J4-8) · TX PB4 (J4-7)	ترانسیور CAN خارجی لازم است. FDCAN1 از پایهٔ BOOT یعنی PB8 استفاده می‌کند.
ورودی‌های ADC	PA0 (J3-9) … PA7 (J3-16); PC0 (J3-5) … PC5 (J3-18); PB0 (J3-19) … PB2 (J3-21); PB11 … PB15	حالت آنالوگ تحمل 5 V ندارد.
خروجی‌های DAC	DAC1_OUT1 PA4 (J3-13) · DAC1_OUT2 PA5 (J3-14) · DAC2_OUT1 PA6 (J3-15)	
TIM1 (پیشرفته)	CH1–CH3 PA8 (J4-16), PA9 (J4-15), PA10 (J4-14) CH1N–CH3N PA7 (J3-16), PB0 (J3-19), PB1 (J3-20) · BKIN PA6 (J3-15)	PWM سه‌فاز با dead time و ورودی break.
TIM8 (پیشرفته)	CH1–CH4 PC6 (J4-20), PC7 (J4-19), PC8 (J4-18), PC9 (J4-17)	
TIM15/16/17	A1/A2 PA8/PA9 · B1 PA10 · C1/C2 PB12/PB13 · D1/D2 PB14/PB15 · E1/E2 PC8/PC9 · F1/F2 PB6/PC7	خروجی‌های تایمر با دقت بالا در STM32G474.
TIM2 · TIM3 · TIM4	TIM2 CH1–CH4 PA0–PA3 · TIM3 CH1–CH4 PA6, PA7, PB0, PB1 · TIM4 CH1–CH4 PB6–PB9	تایمرهای همه‌منظوره (TIM2 32 بیتی است).
دیباگ · خروجی کلاک	SWO PB3 (J4-8) · MCO PA8 (J4-16)	SWO به یک سیم تا پروب نیاز دارد (7.3).
پایه‌های بیدارکردن	WKUP1 PA0 (J3-9) · WKUP2 PC13 (J3-4) · WKUP4 PA2 (J3-11) · WKUP5 PC5 (J3-18)	بیدارکردن از Standby / Shutdown.

نگاشت‌های منتخب و پرکاربرد. همیشه آن‌ها را در STM32CubeMX یا جدول alternate function دیتاشیت STM32G474 تأیید کنید. نکات هر پریفرال در بخش 10 آمده است.

10. راهنمای پریفرال‌ها

راهنمای پریفرال‌به‌پریفرال برای STM32G474RBT6 روی این برد. هر بخش پایه‌های روی هدر، تنظیمی که جواب می‌دهد و دام‌هایی را که بیشترین وقت را می‌گیرند فهرست می‌کند. کدها از HAL استفاده می‌کنند؛ نام handleها را CubeMX می‌دهد. فهرست پایه‌ها همان راهنمای مسیردهی در 9.4 را تکرار می‌کند: پیش از سیم‌کشی آن‌ها را در CubeMX تأیید کنید.

10.1 GPIO و EXTI

- **اول کلاک.** پیش از دست‌زدن به یک پایه، کلاک پورت را فعال کنید (CubeMX این کار را می‌کند؛ با رجیستر از RCC_AHB2ENR استفاده کنید).
- **حالت ریست.** پس از ریست همهٔ پایه‌ها آنالوگ هستند، به‌جز پایه‌های دیباگ (PA13، PA14، PA15، PB3، PB4). پایه‌های استفاده‌نشده جریانی نمی‌کشند و ورودی‌های شناور نمی‌توانند نوسان کنند.
- **سرعت.** برای LED و سیگنال‌های کند Low و فقط برای باس‌های سریع مثل SPI بالاتر از 10 MHz از High یا Very high استفاده کنید. لبه‌های سریع‌تر یعنی EMI و overshoot بیشتر.
- **نوشتن اتمیک.** HAL_GPIO_WritePin از BSRR استفاده می‌کند و در برابر وقفه امن است؛ read-modify-write روی ODR این‌طور نیست.
- **سطح تعریف‌شده برای ورودی‌ها.** از پول‌آپ یا پول‌داون داخلی (چند ده کیلواهم) یا مقاومت خارجی استفاده کنید: ورودی شناور نویز می‌گیرد.

```
void HAL_GPIO_EXTI_Callback(uint16_t pin)      /* one callback for all EXTI lines */
{
  if (pin == GPIO_PIN_0) { /* PA0, PB0 or PC0: whichever port owns line 0 */ }
}
```

★ **نکتهٔ طلایی.** خطوط EXTI بر اساس شمارهٔ پایه مشترک‌اند. خط n فقط به پایهٔ n یک پورت تعلق دارد؛ پس PA0 (J3-9)، PB0 (J3-19) و PC0 (J3-5) نمی‌توانند هم‌زمان ورودی وقفه باشند؛ برای PA8 / PB8 / PC8 و همهٔ شماره‌های دیگر هم همین‌طور. خطوط 5 تا 9 یک بردار مشترک (EXTI9_5_IRQn) و خطوط 10 تا 15 بردار EXTI15_10_IRQn را به اشتراک می‌گذارند، پس پایه را داخل callback بررسی کنید. دکمه‌ها را با 10 تا 20 ms نرم‌افزار یا فیلتر RC debounce کنید و ISR را به گذاشتن یک flag محدود نگه دارید.

10.2 UART، USART و LPUART

پایه‌های روی برد.
USART1 TX PA9 (J4-15), RX PA10 (J4-14) · USART2 TX PA2 (J3-11), RX PA3 (J3-12) · USART3 TX PB10 (J3-22), RX PB11 (J3-23) · UART4 TX PC10 (J4-12), RX PC11 (J4-11) · UART5 TX PC12 (J4-10), RX PD2 (J4-9) · LPUART1 TX PC1 (J3-6), RX PC0 (J3-5)

- **سیم‌کشی.** سطح TTL 3.3 V، TX به RX به‌صورت ضربدری و GND مشترک. از مبدل USB-سریال 3.3 V استفاده کنید، هرگز از سطح RS-232.
- **کلاک هسته.** هر USART می‌تواند از PCLK، SYSCLK، HSI16 یا LSE کار کند. HSI16 نرخ baud را حتی اگر بعداً PLL را عوض کنید ثابت نگه می‌دارد؛ LPUART1 روی LSE (X2) نرخ 9600 baud و بیدارشدن از Stop را ممکن می‌کند.
- **دقت baud.** با oversampling ×16 مقدار BRR = f_clk / baud است، پس جدول زیر در 170 MHz صدق می‌کند. مجموع خطای دو سر را زیر حدود 2 % نگه دارید. حداکثر نرخ 10.6 Mbit/s است (با oversampling ×8 برابر 21.25 Mbit/s).
- **دریافت.** برای فریم‌های با طول متغیر از DMA حلقوی همراه با وقفهٔ idle-line استفاده کنید. در خطای overrun گیرنده را در HAL_UART_ErrorCallback دوباره مسلح کنید.

نرخ baud	9600	115200	460800	921600	1 M	2 M	5 M
BRR (÷16)	17708	1476	369	184	170	85	34
خطا	+0.002 %	−0.02 %	−0.02 %	+0.25 %	0 %	0 %	0 %

```
int __io_putchar(int ch)                                  /* printf over USART1 */
{ HAL_UART_Transmit(&huart1, (uint8_t *)&ch, 1, 10); return ch; }

HAL_UARTEx_ReceiveToIdle_DMA(&huart1, rx_buf, sizeof rx_buf);   /* variable-length frames */
void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *h, uint16_t size) { /* size bytes arrived */ }
```

★ **نکتهٔ طلایی.** RS-485 بدون پایهٔ جهت. USART یک خروجی سخت‌افزاری driver-enable دارد: USART2_DE روی PA1 (J3-10) و USART3_DE روی PB14 (J4-22) (عملکرد پایه RTS_DE). زمان‌های assertion و de-assertion را در تنظیمات USART طوری بگذارید که ترانسیور پیش از اولین بیت فعال و بعد از آخرین بیت آزاد شود.

★ **نکتهٔ طلایی.** printf با float در newlib-nano چیزی چاپ نمی‌کند، مگر اینکه گزینهٔ لینکر «Use float with printf» (-u _printf_float) فعال باشد. PA9 و PA10 کانال‌های I²C2_SCL و TIM1 را هم دارند؛ برای هر پایه یک عملکرد انتخاب کنید.

10.3 SPI

پایه‌های روی برد.
SPI1 SCK PA5 (J3-14), MISO PA6 (J3-15), MOSI PA7 (J3-16), NSS PA4 (J3-13) · SPI2 SCK PB13 (J4-23), MISO PB14 (J4-22), MOSI PB15 (J4-21), NSS PB12 (J4-24) · SPI3 SCK PC10 (J4-12), MISO PC11 (J4-11), MOSI PC12 (J4-10), NSS PA15 (J4-13)

- **انتخاب تراشهٔ نرم‌افزاری.** در CubeMX گزینهٔ NSS = Software را بگذارید و CS هر slave را با یک خروجی GPIO معمولی بزنید. ساده‌تر است و با چند slave کار می‌کند.
- **مد.** CPOL و CPHA را با دیتاشیت slave تطبیق دهید (مد 0 رایج‌ترین است). مد اشتباه به شکل بیت‌های جابه‌جاشده یا درهم دیده می‌شود.
- **سرعت.** از 10 MHz یا کمتر شروع کنید، بعد با تنظیم سرعت پایه روی High یا Very high آن را بالا ببرید. SCK، MOSI و MISO را کوتاه و با GND مشترک نگه دارید؛ برای کلاک‌های سریع 22 تا 47 Ω سری در سمت master بگذارید.
- **DMA.** برای انتقال‌های طولانی از HAL_SPI_TransmitReceive_DMA استفاده کنید و CS را فقط در callback پایان انتقال رها کنید.
- **کد سطح رجیستر.** برای فریم‌های 8 بیتی FRXTH (آستانهٔ RXNE) را بگذارید و DR را 8 بیتی بخوانید، وگرنه RXNE منتظر 16 بیت می‌ماند.

پیش‌تقسیم	2÷	4÷	8÷	16÷	32÷	64÷	128÷	256÷
SCK در 170 MHz (MHz)	85	42.5	21.25	10.63	5.31	2.66	1.33	0.66

```
HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_RESET);
HAL_SPI_TransmitReceive(&hspi1, tx, rx, len, 10);
HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_SET);       /* CS idles high */
```

★ **نکتهٔ طلایی.** هر پایه پس از ریست آنالوگ است، پس خطوط CS هنگام بوت‌شدن MCU شناور می‌مانند و یک slave ممکن است کلاک تصادفی ببیند. روی هر خط CS یک پول‌آپ 10 kΩ بگذارید. پایه‌های NSS و SCK در SPI1 (PA4، PA5) خروجی‌های DAC1 هم هستند: اگر هر دو را لازم دارید، SPI1 را به PB3 / PB4 / PB5 ببرید یا از SPI2 استفاده کنید.

10.4 I²C

پایه‌های روی برد.
I²C1 SCL PB6 (J4-5), SDA PB7 (J4-4) · I²C2 SCL PA9 (J4-15), SDA PA8 (J4-16) · I²C3 SCL PC8 (J4-18), SDA PC9 (J4-17)

- **پول‌آپ.** خطوط I²C از نوع open-drain هستند: به پول‌آپ خارجی به +3.3V نیاز دارند. بخش دیتاشیت هیچ پول‌آپی روی پایه‌های هدر نشان نمی‌دهد، پس خودتان اضافه کنید یا از ماژول سنسوری استفاده کنید که پول‌آپ دارد.
- **زمان‌بندی.** بگذارید CubeMX مقدار TIMINGR را برای کلاک واقعی هسته حساب کند. اگر بعداً SYSCLK، PCLK1 یا منبع کلاک I²C را عوض کردید، آن را دوباره تولید کنید.
- **آدرس.** HAL شکل 8 بیتی را می‌خواهد: آدرس 7 بیتی یک بیت به چپ شیفت‌شده (0x68 می‌شود 0xD0).
- **گیرکردن باس.** اگر پس از ریست در میانهٔ یک انتقال، SDA پایین مانده است، SCL را به GPIO open-drain تبدیل کنید، تا 9 بار آن را بالا و پایین ببرید، یک STOP بفرستید و بعد پریفرال را دوباره مقداردهی کنید.
- **فقط 3.3 V.** برای قطعات 5 V از مبدل سطح ولتاژ استفاده کنید.

حالت	SCL	حداکثر زمان صعود	پول‌آپ معمول
Standard	100 kHz	1000 ns	4.7 kΩ
Fast	400 kHz	300 ns	2.2 تا 3.3 kΩ
Fast-mode Plus	1 MHz	120 ns	1 تا 1.5 kΩ، با فعال‌بودن درایو +Fm (HAL_I2CEx_EnableFastModePlus)

```
for (uint8_t a = 1; a < 127; a++)                              /* I2C scanner */
  if (HAL_I2C_IsDeviceReady(&hi2c1, a << 1, 2, 5) == HAL_OK) printf("found 0x%02X\r\n", a);
```

★ **نکتهٔ طلایی.** پول‌آپ‌ها را از روی ظرفیت خازنی باس انتخاب کنید: $R \le t_r / (0.85 \times C_{bus})$ و $R \ge (3.3\ V - 0.4\ V) / 3\ mA \approx 1\ k\Omega$. ماژول‌ها اغلب پول‌آپ خودشان را دارند (معمولاً 4.7 یا 10 kΩ) و این مقاومت‌ها موازی می‌شوند: سه ماژول با 10 kΩ فقط 3.3 kΩ روی باس باقی می‌گذارند. I²C1 را روی PB7 / PB6 ترجیح دهید: PB9 / PB8 جامپر BOOT را روی خط کلاک می‌گذارد.

10.5 FDCAN

پایه‌های روی برد.
FDCAN2 RX PB5 (J4-6), TX PB6 (J4-5) (توصیه‌شده) · FDCAN1 RX PB8 (J4-3), TX PB9 (J3-3) · FDCAN3 RX PB3 (J4-8), TX PB4 (J4-7)

- **ترانسیور.** پایه‌های MCU در سطح منطقی هستند: ترانسیور CAN لازم است و برد ندارد. برای CAN FD ترانسیوری انتخاب کنید که برای آن تأیید شده باشد (مثلاً TJA1051T/3 یا MCP2562FD که با منطق 3.3 V قابل استفاده‌اند).
- **باس.** مقاومت ترمینیشن 120 Ω فقط در دو انتهای باس، سیم زوج‌تابیده و مرجع زمین مشترک بین گره‌ها.
- **فیلترها.** پس از ریست پریفرال هیچ فریمی را نمی‌پذیرد: یک فیلتر تنظیم کنید یا با HAL_FDCAN_ConfigGlobalFilter پذیرش فریم‌ها در FIFO 0 را فعال کنید.
- **آزمون روی میز.** مد FDCAN_MODE_INTERNAL_LOOPBACK پریفرال را بدون هیچ ترانسیوری آزمایش می‌کند.
- **تنظیمات یکسان در همه‌جا.** همهٔ گره‌ها باید روی CAN کلاسیک یا FD، نرخ بیت و نقطهٔ نمونه‌برداری توافق داشته باشند.

نرخ بیت (کلاک هستهٔ 16 MHz)	پیش‌تقسیم	tq در هر بیت	Seg1	Seg2	نقطهٔ نمونه‌برداری
1 Mbit/s	1	16	13	2	87.5 %
500 kbit/s	2	16	13	2	87.5 %
250 kbit/s	4	16	13	2	87.5 %
125 kbit/s	8	16	13	2	87.5 %
فاز داده در FD، 2 Mbit/s	1	8	6	1	87.5 %

```
HAL_FDCAN_ConfigGlobalFilter(&hfdcan2, FDCAN_ACCEPT_IN_RX_FIFO0, FDCAN_ACCEPT_IN_RX_FIFO0,
                             FDCAN_REJECT_REMOTE, FDCAN_REJECT_REMOTE);
HAL_FDCAN_Start(&hfdcan2);
HAL_FDCAN_ActivateNotification(&hfdcan2, FDCAN_IT_RX_FIFO0_NEW_MESSAGE, 0);
```

★ **نکتهٔ طلایی.** از کریستال 16 MHz به‌عنوان کلاک هستهٔ FDCAN استفاده کنید (RCC → FDCAN clock source = HSE). کریستال بسیار دقیق‌تر از کلاک‌های مشتق‌شده از PLL یا RC است و 16 MHz برای همهٔ نرخ‌های استاندارد دقیقاً به 16 واحد زمانی (time quantum) در هر بیت تقسیم می‌شود (جدول بالا). Sync یک tq است و Seg1 بخش انتشار (propagation) را هم شامل می‌شود؛ از SJW برابر 1 تا 2 استفاده کنید.

10.6 تایمرها و PWM

پایه‌های روی برد.
TIM1 CH1–CH3 PA8 (J4-16), PA9 (J4-15), PA10 (J4-14) · CH1N–CH3N PA7 (J3-16), PB0 (J3-19), PB1 (J3-20) · BKIN PA6 (J3-15) · TIM8 CH1–CH4 PC6 (J4-20), PC7 (J4-19), PC8 (J4-18), PC9 (J4-17) · TIM2 CH1–CH4 PA0–PA3 · TIM3 PA6, PA7, PB0, PB1 · TIM4 PB6–PB9

- **کلاک تایمر.** وقتی تقسیم‌کنندهٔ APB روی ÷1 است (6.2)، تایمرها با 170 MHz کار می‌کنند. با تقسیم‌کنندهٔ ÷2 یا بیشتر، کلاک تایمر دو برابر PCLK است.
- **ریاضی PWM.** $f_PWM = f_timer / ((PSC + 1) \times (ARR + 1))$؛ وضوح تقریباً $\log_2(ARR + 1)$ بیت است. حالت center-aligned برای همان ARR، فرکانس را نصف می‌کند.

- **Preload.** پیش‌بارگذاری ARR و CCR را فعال نگه دارید تا تغییرها در لحظهٔ update event اعمال شوند و خروجی هیچ‌وقت glitch نداشته باشد.
- **نمونهٔ سرو.** PSC = 169 (تیک 1 MHz)، ARR = 19999 (50 Hz)، تا CCR = 1000 2000 برای پالس 1 تا 2 ms.
- **انکودر.** TIM3 (PA7 / PA6) یا TIM4 (PB7 / PB6) در حالت encoder با فیلتر ورودی فعال؛ انکودرهای open-collector پول‌آپ می‌خواهند (پول‌آپ‌های داخلی ضعیف‌اند).
- **Input capture.** TIM2 عرض 32 بیت دارد: برای دوره‌های طولانی و اندازه‌گیری PWM-input روی PA0 تا PA3 از آن استفاده کنید.

فرکانس PWM	20 kHz	50 kHz	100 kHz	500 kHz	1 MHz
تعداد شمارش در هر دوره (ARR + 1)	8500	3400	1700	340	170
وضوح	≈ 13.1 bit	≈ 11.7 bit	≈ 10.7 bit	≈ 8.4 bit	≈ 7.4 bit

```
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);           /* PA8  */
HAL_TIMEx_PWMN_Start(&htim1, TIM_CHANNEL_1);        /* PA7, complementary with dead time */
__HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, (__HAL_TIM_GET_AUTORELOAD(&htim1) + 1) / 2);   /* 50 % */
```

★ **نکتهٔ طلایی.** تایمرهای پیشرفته (TIM1، TIM8) برای طبقات قدرت. هر تیک dead-time در 170 MHz حدود 5.9 ns است؛ dead time را از دیتاشیت گیت‌درایور و MOSFET انتخاب کنید (اغلب 100 تا 500 ns). ورودی BKIN) break روی PA6، J3-15) و حالت‌های بیکار خروجی را پیش از فعال‌کردن خروجی‌ها تنظیم کنید، هم کانال PWM و هم PWMN را استارت کنید و MOE را فقط وقتی set نگه دارید که قرار است طبقهٔ قدرت کار کند. پایه‌های PA7 / PB0 / PB1 در TIM1 پایه‌های ADC و SPI هم هستند: از قبل برنامه‌ریزی کنید.

★ **نکتهٔ طلایی.** زمان‌سنجی میکروثانیه‌ای بدون تایمر: شمارندهٔ سیکل Cortex-M4 را فعال کنید. در 170 MHz، دقیقاً 170 سیکل برابر 1 µs است. HRTIM1 (STM32G474) خیلی جلوتر می‌رود، با دقت 184 ps روی PA8 تا PA10، PB12 تا PB15 و PC6 تا PC9: برای تبدیل توان دیجیتال از ویزارد HRTIM در CubeMX استفاده کنید.

```
CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;   /* enable the DWT unit */
DWT->CYCCNT = 0;
DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;               /* DWT->CYCCNT counts CPU cycles */
```

10.7 ADC

پایه‌های روی برد.

PA0 (J3-9) … PA7 (J3-16) · PC0 (J3-5) … PC5 (J3-18) · PB0 (J3-19) … PB2 (J3-21) · PB11 …
PB15 · Internal: temperature sensor, VBAT/3, VREFINT

- **کلاک.** کلاک ADC باید حداکثر 60 MHz بماند. در CubeMX از کلاک ناهمگام (asynchronous) با 4÷ SYSCLK (42.5 MHz) استفاده کنید.
- **زمان تبدیل.** برای 12 بیت، t_conv = (sampling time + 12.5) سیکل ADC است. جدول زیر کلاک ADC برابر 42.5 MHz را فرض می‌کند.
- **زمان نمونه‌برداری.** آن را از روی مقاومت منبع با جدول دیتاشیت انتخاب کنید: منبع با امپدانس بالا به 92.5 سیکل یا بیشتر نیاز دارد؛ آن را با آپ‌امپ بافر کنید یا یک خازن روی پایه بگذارید.
- **کالیبراسیون** را پس از هر روشن‌شدن و پیش از اولین تبدیل انجام دهید.
- **Oversampling.** oversampler سخت‌افزاری (تا 256× همراه با شیفت به راست) نتیجه‌های تا 16 بیت می‌دهد و نویز را بدون بار روی CPU میانگین می‌گیرد.
- **هرگز از VREF+ فراتر نروید.** پایه‌های آنالوگ نباید بیش از 3.3 V ببینند؛ سیگنال‌های بزرگ‌تر را با تقسیم‌کنندهٔ ولتاژ کوچک کنید و مقاومت منبع آن را در نظر بگیرید.

زمان نمونه‌برداری (سیکل)	2.5	6.5	12.5	24.5	47.5	92.5	247.5	640.5
زمان تبدیل (µs)	0.35	0.45	0.59	0.87	1.41	2.47	6.12	15.4

```
HAL_ADCEx_Calibration_Start(&hadc1, ADC_SINGLE_ENDED);        /* once after power-up */
HAL_ADC_Start_DMA(&hadc1, (uint32_t *)adc_buf, N);            /* circular DMA, half/full callbacks */
HAL_TIM_Base_Start(&htim6);                                   /* TIM6 TRGO triggers every conversion */
```

★ **نکتهٔ طلایی.** نمونه‌برداری را با تایمر انجام دهید، نه با حلقه. ADC را از TRGO یک تایمر ماشه کنید (TIM6 برای نمونه‌برداری ساده، تایمر پیشرفته برای کنترل موتور) و داده را با DMA حلقوی جابه‌جا کنید: نرخ نمونه‌برداری آن‌وقت دقیق و مستقل از CPU است. برای سنجش جریان از کانال‌های injected استفاده کنید که در مرکز دورهٔ PWM و دور از لبه‌های سوئیچینگ ماشه می‌شوند. سنسور دما و VBAT را با زمان نمونه‌برداری بلند (چند میکروثانیه) بخوانید و با مقادیر کالیبراسیون کارخانه تبدیل کنید (ماکروهای LL: __LL_ADC_CALC_TEMPERATURE و __LL_ADC_CALC_VREFANALOG_VOLTAGE).

10.8 DAC

پایه‌های روی برد.

DAC1_OUT1 PA4 (J3-13) · DAC1_OUT2 PA5 (J3-14) · DAC2_OUT1 PA6 (J3-15)

- **بافر خروجی.** با بافر روشن، خروجی بارهای مقاومتی کوچک را می‌راند اما تقریباً از 0.2 V تا VDDA − 0.2 V نوسان می‌کند (حدود دقیق را در دیتاشیت ببینید). با بافر خاموش، خروجی یک گره با امپدانس بالاست: آن را بیرونی بافر کنید.
- **شکل موج.** یک جدول جست‌وجو پر کنید، DAC را با ماشهٔ تایمر (TIM6 یا TIM7) راه بیندازید و بگذارید DMA حلقوی آن را تغذیه کند. نرخ نمونه‌برداری همان نرخ update تایمر است؛ فرکانس شکل موج برابر نرخ نمونه‌برداری تقسیم بر طول جدول است.
- **مرجع.** مرجع DAC همان VREF+ = 3.3 V از LDO است، پس دقت مطلق آن فقط به اندازهٔ همین ریل است؛ مدارهای نسبتی را از همان 3.3 V تغذیه کنید.
- **تریم آفست.** وقتی از بافر خروجی استفاده می‌کنید، هنگام راه‌اندازی HAL_DACEx_SelfCalibrate را اجرا کنید.
- **فیلتر.** برای صدا یا سطوح آنالوگ نرم، بعد از پایه یک فیلتر پایین‌گذر RC اضافه کنید.

```
HAL_DAC_Start_DMA(&hdac1, DAC_CHANNEL_1, (uint32_t *)sine, 64, DAC_ALIGN_12B_R);
HAL_TIM_Base_Start(&htim6);                    /* 64 samples per period: f_wave = f_TIM6 / 64 */
```

★ **نکتهٔ طلایی.** یک سینوس 1 kHz از 64 نمونه به ماشهٔ 64 kHz نیاز دارد: با TIM6 روی کلاک 170 MHz از PSC = 0 و ARR = 2655 استفاده کنید (170 MHz / 2656 = 64.01 kHz). پایه‌های PA4 و PA5 در عین حال NSS / SCK SPI1 هستند و PA6 هم TIM1_BKIN و SPI1_MISO است، پس زودتر تصمیم بگیرید کدام عملکرد صاحب آن‌هاست.

10.9 DMA

- **ترتیب راه‌اندازی.** کلاک DMA باید پیش از پریفرالی که از آن استفاده می‌کند روشن شود: MX_DMA_Init() باید پیش از MX_USARTx_UART_Init()، MX_ADCx_Init() و مشابه آن‌ها اجرا شود. ترتیب غلط، علت کلاسیک «DMA هیچ‌وقت شروع نمی‌شود» است.
- **عرض داده.** عرض حافظه و پریفرال باید با داده بخواند: بایت برای UART و SPI 8 بیتی، half-word برای دادهٔ 12 بیتی ADC و DAC. عدم تطابق، خروجی‌ای شبیه نویز می‌دهد.
- **حالت حلقوی با callback.** از callbackهای نیم‌انتقال و انتقال کامل به‌عنوان بافر دوگانه استفاده کنید: یک نیمه را پردازش کنید در حالی که DMA نیمهٔ دیگر را پر می‌کند.
- **مسیردهی و اولویت.** STM32G474 دو کنترلر DMA با 8 کانال و یک DMAMUX دارد که هر درخواست را به هر کانال وصل می‌کند: CubeMX آن‌ها را تخصیص می‌دهد. به جریان‌های حساس به زمان (ADC، DAC) بالاترین اولویت را بدهید.

- **بافرها.** بافرهای DMA را در SRAM عادی که اسکریپت لینکر تعریف کرده نگه دارید، آن‌ها را global یا static بگیرید (هرگز روی پشته‌ای که از بین می‌رود) و تا وقتی DMA صاحب بافر است به آن دست نزنید.

10.10 کامپاراتورها، آپ‌امپ‌ها و شتاب‌دهنده‌های ریاضی

- **کامپاراتورها.** STM32G474 هفت کامپاراتور سریع دارد. خروجی آن‌ها می‌تواند ورودی break تایمر را مستقیماً با سخت‌افزار برانگیزد و یک trip اضافه‌جریان در چند نانوثانیه بدهد، بدون دخالت CPU؛ یک کانال داخلی DAC می‌تواند آستانه را تأمین کند.
- **آپ‌امپ‌ها.** شش آپ‌امپ داخلی می‌توانند به‌صورت دنبال‌کننده یا در حالت گین برنامه‌پذیر (گین تا 64) کار کنند و سیگنال شنت را مستقیم برای ADC تقویت کنند. پایه‌های آن‌ها با پایه‌های ADC و GPIO مشترک‌اند؛ پیش از طراحی PCB جدول pin-definition را برای بستهٔ 64 پین بررسی کنید.
- **CORDIC.** یک واحد سخت‌افزاری برای سینوس، کسینوس، آرک‌تانژانت و جذر: برای تبدیل‌های Park و Clarke در کنترل موتور مفید است. داده ممیز ثابت است (q1.31 یا q1.15) و زاویه‌ها بر π مقیاس می‌شوند.
- **FMAC.** یک موتور فیلتر برای فیلترهای FIR و IIR که با DMA تغذیه می‌شود و در حلقه‌های پردازش سیگنال هسته را سبک می‌کند.

10.11 فریمور دستگاه USB

- **کلاس.** CDC (پورت COM مجازی) ساده‌ترین راه برای گرفتن لاگ و فرمان از طریق کابل Type-C است. HID برای کیبورد و ماوس، DFU برای به‌روزرسانی در میدان و MSC برای ذخیره‌سازی مناسب است.
- **ارسال CDC.** CDC_Transmit_FS تا وقتی بستهٔ قبلی در حال ارسال است USBD_BUSY برمی‌گرداند: به‌جای فراخوانی در حلقهٔ تنگ، از ring buffer و callback پایان ارسال استفاده کنید.
- **پیام اول گم می‌شود؟** دادهٔ ارسال‌شده پیش از بازشدن پورت توسط رایانه دور ریخته می‌شود: پیش از اولین چاپ منتظر خط وضعیت DTR میزبان بمانید.
- **نرخ baud.** نرخی که ترمینال روی پورت COM مجازی تنظیم می‌کند نادیده گرفته می‌شود؛ پیوند همیشه با سرعت USB کار می‌کند.
- **VID / PID.** مقادیر 0x5740 / 0x0483 مقادیر نمایشی ST برای پورت COM مجازی هستند. برای آزمایش خوب‌اند، اما محصول به VID و PID خودش نیاز دارد.
- **حافظه.** اگر دستگاه شناسایی نمی‌شود، اندازهٔ heap و stack در اسکریپت لینکر را بررسی کنید: میان‌افزار USB حافظه تخصیص می‌دهد.

> ★ **نکتهٔ طلایی.** کلاک و شناسایی مجدد تعیین می‌کنند USB چقدر محکم کار می‌کند. HSI48 قفل‌شده با CRS (یا گزینهٔ 168 MHz) را مطابق 6.2 به کار ببرید و هنگام راه‌اندازی از HAL_PCD_DevConnect / HAL_PCD_DevDisconnect استفاده کنید تا رایانه پس از هر دانلود یک اتصال تازه ببیند.

10.12 واچ‌داگ‌ها و مدیریت خطا

- **واچ‌داگ مستقل (IWDG).** از LSI کار می‌کند. با پیش‌تقسیم ÷32 و مقدار reload برابر 999، تایم‌اوت حدود یک ثانیه است. تلرانس LSI زیاد است، پس از IWDG برای گرفتن قفل‌شدن‌ها استفاده کنید، نه برای اندازه‌گیری زمان.
- **واچ‌داگ پنجره‌ای (WWDG).** از کلاک APB کار می‌کند و اگر آن را زودتر از موعد refresh کنید هم MCU را ریست می‌کند، که حلقه‌های فراری را می‌گیرد.
- **متوقف‌کردن هنگام دیباگ.** در بیلدهای debug __HAL_DBGMCU_FREEZE_IWDG() را صدا بزنید، وگرنه واچ‌داگ هر بار که روی breakpoint می‌ایستید MCU را ریست می‌کند.
- **HardFault.** حلقهٔ بی‌پایان پیش‌فرض را رها نکنید. PC ذخیره‌شده در پشته را نگه دارید، بعد PC13 LED را با یک الگو چشمک بزنید یا از طریق SWO چاپ کنید: خطای قابل‌دیدن ساعت‌ها وقت را نجات می‌دهد.
- **Brown-out و علت ریست.** سطح brown-out reset را متناسب با تغذیهٔ خود تنظیم کنید و پرچم‌های ریست (RCC_CSR) را هنگام راه‌اندازی بخوانید تا بفهمید آخرین ریست واچ‌داگ بوده، brown-out یا دکمهٔ NRST.

10.13 کم‌مصرفی و بیدارکردن

- **حالت‌ها.** Sleep فقط CPU را متوقف می‌کند؛ Stop 0 و Stop 1 حافظهٔ RAM را نگه می‌دارند و در چند میکروثانیه بیدار می‌شوند؛ Standby فقط حوزهٔ backup را نگه می‌دارد و از راه ریست بیدار می‌شود؛ Shutdown کم‌مصرف‌ترین حالت است.
- **منابع بیدارکردن.** پایه‌های PA0 (J3-9)، PC13 (J3-4)، PA2 (J3-11) و PC5 (J3-18)، هشدار یا تایمر بیدارکنندهٔ RTC (با کریستال LSE یعنی X2) و LPUART1 روی LSE.
- **پس از Standby** برنامه از ریست شروع می‌شود. پرچم standby (PWR_FLAG_SB) را بررسی و پاک کنید و وضعیت خود را از رجیسترهای backup برگردانید.
- **حالت پایه‌ها.** در Standby پایه‌ها شناور می‌مانند. برای نگه‌داشتن یک سطح از رجیسترهای پیکربندی PWR pull-up و pull-down استفاده کنید (با بیت APC فعال می‌شوند).
- **دیباگ.** پیوند دیباگ در Stop و Standby می‌تواند قطع شود: هنگام توسعه DBG_STOP و DBG_STANDBY را در DBGMCU_CR تنظیم کنید.

```
if (__HAL_PWR_GET_FLAG(PWR_FLAG_SB)) { __HAL_PWR_CLEAR_FLAG(PWR_FLAG_SB); /* woke from Standby */ }
```

★ **نکتهٔ طلایی.** روی این برد میکروآمپرها را محک نزنید. POWER LED، جریان آرامش خود LDO و PC13 LED همگی بیشتر از یک STM32G4 در حال خواب مصرف می‌کنند و جامپر اندازه‌گیری IDD هم مستند نشده است. اعداد کم‌مصرفی MCU را از دیتاشیت ST یا از جریان VBAT با USB جداشده قضاوت کنید؛ عمر باتری برد را با اندازه‌گیری کل برد بسنجید.

11. RTC و باتری پشتیبان

کریستال 32.768 kHz به نام X2 کلاک RTC را می‌دهد و جایگاه CR1220 در سمت پایین، VBAT را تغذیه می‌کند؛ بنابراین RTC و رجیسترهای backup با برداشتن USB به کار خود ادامه می‌دهند.

1. **باتری را در جایگاه بگذارید** (به علامت قطبیت جایگاه دقت کنید).
2. **دسترسی به حوزهٔ backup را فعال کنید،** LSE را راه بیندازید و آن را به‌عنوان کلاک RTC انتخاب کنید (CubeMX: RCC → LSE crystal، منبع کلاک RTC برابر LSE).
3. **زمان را یک‌بار تنظیم کنید** و یک مقدار «جادویی» در یک رجیستر backup بنویسید (HAL_RTCEx_BKUPWrite). در بوت بعدی آن را بخوانید: اگر برابر بود، RTC از قبل در حال کار است و نباید دوباره مقداردهی شود.

هرگز شارژ VBAT را با CR1220 فعال نکنید. STM32G4 می‌تواند VBAT را از طریق یک مقاومت داخلی شارژ کند (بیت PWR_CR4 VBE). CR1220 سلول لیتیومی یک‌بارمصرف است: VBE را روی 0 نگه دارید.

★ **نکتهٔ طلایی.** باتری را زیر نظر بگیرید. کانال ADC داخلی VBAT/3 را بخوانید (HAL: ADC_CHANNEL_VBAT) و وقتی ولتاژ از حدود 2.5 V پایین‌تر رفت به کاربر هشدار بدهید. عمر باتری را با تقسیم ظرفیت بر جریان حوزهٔ backup که در دیتاشیت ST آمده تخمین بزنید.

★ **نکتهٔ طلایی.** دقت زمان و دام‌های API. یک کریستال 32.768 kHz معمولی حدود ±20 ppm دقت دارد، یعنی $20 \times 10^{-6} \times 86\,400\ \text{s} \approx 1.7\ \text{s}$ در روز؛ آن را با smooth calibration (HAL_RTCEx_SetSmoothCalib) تنظیم کنید. همیشه HAL_RTC_GetTime را پیش از HAL_RTC_GetDate صدا بزنید: خواندن زمان رجیسترهای shadow را قفل و خواندن تاریخ آن‌ها را باز می‌کند. برای بیدارشدن‌های دوره‌ای از HAL_RTCEx_SetWakeUpTimer_IT استفاده کنید.

12. چک‌لیست نکات طلایی

پیش از اولین روشن‌کردن

- جامپر BOOT باز
- هیچ منبع تغذیه‌ای به پایه‌های +5V / +3.3V وصل نیست
- بار خارجی در مجموع زیر 50 mA برنامه‌ریزی شده
- قطبیت CR1220 بررسی شده

پیش از برنامه‌ریزی

- GND، SWCLK، SWDIO وصل شده (در صورت نیاز SWO روی PB3)
- Debug = Serial Wire؛ HSE_VALUE = 16000000
- کلاک USB از HSI48 + CRS
- پایه‌های استفاده‌نشده در حالت آنالوگ؛ شارژ VBAT خاموش

پیش از تحویل محصول

- جریان واقعی فریمور در برابر 50 mA اندازه‌گیری شده
- CSS و واچ‌داگ فعال
- تصمیم RDP گرفته شده (هرگز Level 2 روی برد توسعه)
- شناسایی مجدد USB آزمایش شده

پیش‌پرواز پریفرال‌ها

پریفرال	پیش از آزمایش بررسی کنید
GPIO · EXTI	کلاک پورت روشن؛ یک پورت برای هر خط EXTI؛ پول‌آپ یا پول‌داون روی ورودی‌ها؛ محدودیت‌های PB8 و PC13 رعایت شده.
UART	TX و RX ضربدری، GND مشترک، مبدل 3.3 V، خطای baud زیر 1 % (10.2).
SPI	مد و سرعت با slave جور است؛ CS در حالت بیکار بالاست و پول‌آپ 10 kΩ دارد؛ انتقال‌های DMA پیش از بالارفتن CS تمام می‌شوند.
I²C	پول‌آپ خارجی وجود دارد؛ آدرس 7 بیتی به چپ شیفت شده؛ TIMINGR پس از تغییر کلاک دوباره تولید شده.
FDCAN	ترانسیور نصب شده؛ 120 Ω در دو انتهای باس؛ فیلتر تنظیم شده؛ HSE به‌عنوان کلاک هسته؛ زمان‌بندی بیت در همهٔ گره‌ها یکسان.
PWM · TIM1	dead time و break تنظیم شده؛ خروجی‌ها هنگام ریست امن‌اند؛ هر دو کانال PWM و PWMN استارت شده؛ MOE فقط هنگام نیاز set می‌شود.
ADC	کالیبره شده؛ زمان نمونه‌برداری با امپدانس منبع جور است؛ ورودی هرگز بالاتر از 3.3 V نیست؛ ماشهٔ تایمر و DMA آماده است.
DAC	تنظیم بافر با بار جور است؛ تایمر ماشه در حال کار است؛ طول جدول و نرخ نمونه‌برداری فرکانس دلخواه را می‌دهند.
DMA	MX_DMA_Init پیش از راه‌اندازی پریفرال اجرا شده؛ عرض داده‌ها جور است؛ بافرها global یا static هستند.
USB	HSI48 + CRS یا گزینهٔ 168 MHz؛ کابل داده‌دار؛ شناسایی مجدد آزمایش شده؛ heap و stack کافی است.

13. عیب‌یابی

نشانه	علت محتمل	چه کنید
POWER LED خاموش است	کابل یا درگاه خراب، اتصال کوتاه روی یک هدر	کابل و درگاه دیگری امتحان کنید؛ همه‌چیز را از هدرها جدا کنید؛ اتصال کوتاه +5V / +3.3V به GND را بررسی کنید.
رایانه دستگاه USB را نمی‌بیند	کابل فقط‌شارژ؛ کلاک USB تنظیم نشده؛ فریمور اجرا نمی‌شود	از کابل داده‌دار استفاده کنید؛ HSI48 + CRS را تنظیم کنید (6.2)؛ با جامپر BOOT روی DFU امتحان کنید (7.4).
SWD: «no target»	سیم‌کشی، فریمور SWD را غیرفعال کرده، حالت کم‌مصرف	سه سیم و GND را بررسی کنید؛ کلاک SWD را کم کنید؛ با دکمهٔ NRST تحت ریست وصل شوید؛ از طریق DFU پاک‌سازی کنید (7.4).
برد در هر ریست وارد بوت‌لودر می‌شود	جامپر BOOT نصب است یا PB8 بالا کشیده شده	جامپر را بردارید؛ سیم‌کشی خارجی روی PB8 را بررسی کنید.
LED سبز چشمک نمی‌زند	پایه، کلاک یا قطبیت اشتباه	PC13 بالا = روشن؛ کلاک GPIOC را فعال کنید؛ PC13 را خروجی بگذارید.
نرخ baud یا زمان‌بندی‌ها غلط است	عدم تطابق HSE_VALUE یا PLL	HSE_VALUE = 16000000 بگذارید؛ SYSCLK را روی MCO (PA8) بررسی کنید.

نشانه	علت محتمل	چه کنید
سیستم در SystemClock_Config گیر می‌کند	HSE استارت نمی‌شود یا مقادیر PLL بد است	با 6.2 مقایسه کنید؛ X1 و خازن‌های 10 pF آن را بررسی کنید؛ هنگام آزمایش یک fallback روی HSI16 نگه دارید.
UART آشغال چاپ می‌کند	عدم تطابق baud یا کلاک، RX / TX جابه‌جا، مبدل 5 V یا RS-232	جدول 10.2 را ببینید؛ TX و RX را عوض کنید؛ SYSCLK را روی MCO بررسی کنید؛ از مبدل 3.3 V استفاده کنید.
قطعهٔ I²C پیدا نمی‌شود	پول‌آپ نیست، آدرس شیفت نشده، پایه اشتباه، TIMINGR کهنه	پول‌آپ 2.2 تا 4.7 kΩ اضافه کنید؛ از 1 << address استفاده کنید؛ اسکنر (10.4) را اجرا کنید.
SPI مقدار 0xFF یا 0x00 برمی‌گرداند	مد اشتباه، CS فعال نشده، MISO وصل نیست، slave تغذیه ندارد	CPOL / CPHA و CS را بررسی کنید؛ SCK را با اسیلوسکوپ ببینید؛ پول‌آپ CS بگذارید (10.3).
خروجی PWM تایمر TIM1 نیست	MOE یا break فعال، عملکرد پایه اشتباه، کانال مکمل استارت نشده	هم PWM و هم PWMN را استارت کنید؛ BKIN را بررسی کنید؛ alternate function را در CubeMX تأیید کنید (10.6).
CAN فعالیتی ندارد یا خطا می‌دهد	ترانسیور یا ترمینیشن نیست، زمان‌بندی بیت غلط، فیلتر فریم‌ها را رد می‌کند	بخش 10.5 را بررسی کنید: ترانسیور، 120 Ω، زمان‌بندی 16 tq، فیلتر سراسری؛ internal loopback را امتحان کنید.
انتقال‌های DMA آشغال می‌دهند	عدم تطابق عرض داده، ترتیب راه‌اندازی، بازنویسی بافر	عرض‌ها را جور کنید؛ MX_DMA_Init() را اول صدا بزنید؛ حالت حلقوی با callbackهای نیمه / کامل استفاده کنید (10.9).
ADC مقدار 0 یا 4095 می‌خواند	پایه در حالت آنالوگ نیست، کانال اشتباه، ورودی بالاتر از 3.3 V	پایه را آنالوگ تنظیم کنید؛ کانال را بررسی کنید؛ ورودی را به VREF+ محدود کنید (10.7).
مقدارهای ADC می‌پرند یا نویزی‌اند	نویز مرجع، امپدانس منبع	کالیبره کنید؛ نمونه‌برداری را بلندتر کنید؛ oversampling بزنید؛ با تایمر ماشه کنید؛ از VREFINT استفاده کنید (10.7).
RTC با قطع تغذیه زمان را گم می‌کند	باتری نیست، LSE استارت نشده، حوزهٔ backup ریست شده	CR1220 بگذارید؛ LSE را راه بیندازید؛ از بررسی رجیستر backup استفاده کنید (بخش 11).
برد داغ می‌شود یا ریست می‌خورد	بار بیش از 50 mA	بارها را بردارید؛ جریان را اندازه بگیرید؛ از تغذیهٔ جداگانه استفاده کنید (بخش 5).
MCU هنگام دیباگ ریست می‌شود	IWDG روی breakpoint هنوز کار می‌کند	در بیلدهای debug __HAL_DBGMCU_FREEZE_IWDG() را صدا بزنید (10.12).

14. مراجع و تاریخچهٔ نسخه‌ها

- دیتاشیت AH_G474، نسخهٔ V0.1 (همین برد).
- دیتاشیت STM32G474xB/C/E و RM0440 (مرجع سری STM32G4)، ST.
- AN2606: حالت بوت حافظهٔ سیستم در میکروکنترلرهای STM32، ST.
- PM0214: راهنمای برنامه‌نویسی Cortex-M4 در STM32، ST.
- مستندات STM32CubeIDE، STM32CubeMX و STM32CubeProgrammer و توضیح درایورهای HAL و LL سری STM32G4، ST.

نسخهٔ سند	تاریخ	سخت‌افزار	تغییرات
V0.1	نوامبر 2026	V0.1	انتشار اولیه.
V0.2	نوامبر 2026	V0.1	افزودن راهنمای پریفرال‌ها (بخش 10) همراه با جدول و کد، نکات RTC، چک‌لیست پیش‌پرواز پریفرال‌ها و ردیف‌های بیشتر در عیب‌یابی؛ ویرایش فارسی.

@AliasgharZ01 — اطلاعات برد در این راهنما از دیتاشیت AH_G474 (نسخهٔ V0.1) گرفته شده است. اطلاعات و نکات میکروکنترلر به مستندات ST ارجاع دارند و باید با آخرین نسخهٔ آن‌ها تطبیق داده شوند. کشیدن بیش از 50 mA از برد به رگولاتور ولتاژ آسیب می‌زند. ایمنی مدارهای خارجی متصل به برد بر عهدهٔ کاربر است.